MALLA REDDY UNIVERSITY
Department of Cybersecurity, ECE & IoT
Maisammaguda, Dulapally, Hyderabad, Telangana — 500100

CodeGalaxy: An Agentic AI Based Interactive Programming Learning Platform

See what your program does at run time — real execution, AI diagnosis, live visualisation and gamified progress in one browser workspace.

5	3	40	30
Languages (Python, JS, Java, C, C++)	LLM Providers in Fallback Chain	REST Endpoints across 9 Modules	Frontend Pages

PROJECT TEAM

Harsh Kumar
2311CS040064
Dept. of Cybersecurity, ECE & IoT

Abhishek Pratap Kushwaha
2311CS040003
Dept. of Cybersecurity, ECE & IoT

Aditya Raj
2311CS040006
Dept. of Cybersecurity, ECE & IoT

ABSTRACT

CodeGalaxy is a full-stack, browser-based platform for beginners who understand syntax but struggle to see how a program actually behaves at run time. A Monaco-based multi-language workspace runs code on a local execution engine; when a run fails, the source, language and stderr are sent to a multi-provider LLM router (Gemini → Groq → Ollama) that returns a structured, step-by-step diagnosis instead of the fixed program. Around this core sit a zero-infrastructure TF-IDF document Q&A pipeline, hand-built algorithm visualisers, a gamified "solar-system" skill map with XP, streaks and badges, and an AI technical-interview simulator. This poster reports the system as it exists in source, including where the design documents describe more autonomy than the code yet implements.

RESEARCH GAPS ADDRESSED

- **No visibility into run-time state:** online judges return pass/fail or a stack trace, not why the output was produced.
- **Isolated AI help:** chat copilots are not tied to an execution sandbox, a progress model or a curriculum.
- **Execution-blind LMS:** classroom systems track grades but never run or reason about code.
- **Spoiler answers:** rewriting the learner's code teaches less than guided diagnosis of the actual error.

PIPELINE: METHODOLOGY

1 Write — Monaco editor, 5 languages
In-browser editor for Python, JS, Java, C and C++. A pre-assessment and AI-generated learning path pick the next challenge.

→ **2 Execute** — Local engine, 8 s limit
Source is written to a UUID temp file and run via gcc / g++ / python / node / javac with an 8 s limit and 10 MB output cap.

→ **3 AI Diagnose** — Gemini → Groq → Ollama
On a failing run only, source + stderr go to the LLM router, which returns a step-by-step diagnosis without the fixed code.

→ **4 Visualise** — React visualisers
Hand-built components animate sorting, linked lists and trees with playback controls and live complexity.

→ **5 Reward & Adapt** — XP · badges · skill map
Passing runs award XP, levels and badges; a failed quiz triggers a remedial explanation instead of unlocking the next node.

LAYERED SYSTEM ARCHITECTURE

Presentation	React 18 · TypeScript · Tailwind · Radix UI · Framer Motion
Application	Node.js · Express 5 · JWT auth · 9 route modules
Execution	child_process · gcc / g++ / python / node / javac · 8 s limit
AI Orchestration	Gemini → Groq → Ollama → mock fallback; TF-IDF RAG
Data & State	MongoDB Atlas · Mongoose 9 · in-memory document store

END-TO-END LEARNER WORKFLOW

1 Sign up / Login · 2 AI Pre-assessment · 3 Learning Path
4 Write Code · 5 Compile & Run · 6 AI Error Repair
7 Visualise · 8 Earn XP & Badges · 9 Interview Prep

VERIFIED IN SOURCE

- ✓ stderr alone is not treated as failure
- ✓ 11 JSON generators, each with a mock fallback
- ✓ TF-IDF RAG: ~900-char chunks, 150 overlap
- ✓ Failed quiz → adaptive remedial help
- ✓ 7 interview-prep endpoints, voice in/out

ANALYTICS: THE CODEBASE IN NUMBERS

Overall: lines of code



Frontend	22,459	78%
Backend	6,310	22%

Frontend: lines by area



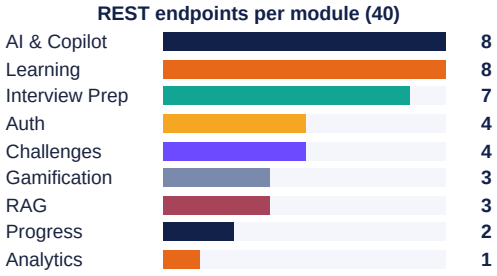
Components	11,208	50%
Pages (30)	9,138	41%
Hooks	550	2%
Shell, utils, types	1,563	7%

Backend: lines by layer



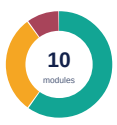
Services	1,942	31%
Controllers	1,425	23%
Models	407	6%
Utils	205	3%
Routes	149	2%
Middleware	38	1%
Server & scripts	2,144	34%

REST endpoints per module (40)



Counts are JS/TS/JSX lines in the repository, excluding node_modules, build output and temp folders. "Server, seed & scripts" covers server bootstrap, seed and debug scripts.

IMPLEMENTATION STATUS (VERIFIED IN SOURCE)



Working in live app	6	60%
Implemented, not wired	3	30%
Not yet built	1	10%

- LIVE Auth (JWT + bcrypt)
- LIVE Code execution engine
- LIVE Multi-provider LLM router
- LIVE TF-IDF document RAG
- LIVE Gamification engine
- LIVE Interview simulator
- UNWIRED Agent loop (planner-executor)
- UNWIRED Prompt-chain module
- UNWIRED Qdrant semantic search
- PLANNED Docker / container sandbox

LLM FALLBACK: WHO ANSWERS?

Gemini	Groq	Ollama	Answered by
✓ on	✓ on	✓ on	**Gemini** first try
✗ off	✓ on	✓ on	**Groq** after 1 failed attempt
✗ off	✗ off	✓ on	**Ollama (local)** after 2 failed attempts
✗ off	✗ off	✗ off	**Mock generator** valid canned content, no error

Two live model-deprecation incidents (Gemini and Groq model names) were absorbed by this chain, and masked for a time until a start-up diagnostic was added.

"FOUR AGENTS": DESIGN VS. IMPLEMENTATION

Agent	As designed	Implemented today
Code Analysis	Detects syntax/logic errors, suggests fixes	One JSON call (`analyzeCodeFeedback` / `debugCodeError`) via the shared router
Execution Visualization	Real-time visuals of variables, loops, structures	Client-side React visualisers; no server-side AI
Learning Assistant	Answers questions, personalised guidance	`generateChatReply` (+ stream) and TF-IDF document Q&A
Assessment	Evaluates skills, generates challenges	`generateSkillProfile`, interview quiz, `evaluateInterviewCode`

IMPLEMENTATION: WORKING PROTOTYPE

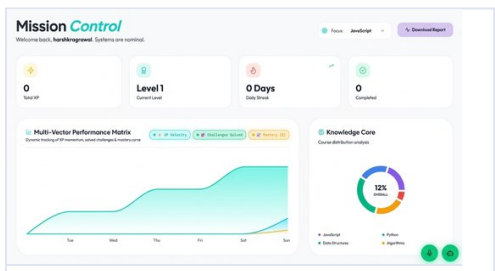


Fig. 1 Mission Control: XP, level, streak, performance chart and knowledge-core donut

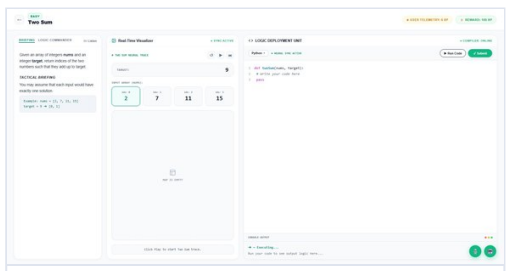


Fig. 2 Challenge workspace: brief, real-time visualiser, editor and console



Fig. 3 DSA visualiser: step-by-step sorting animation

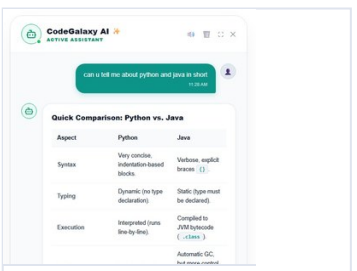


Fig. 4 AI assistant with text/voice input and Markdown tables

TECHNOLOGY STACK & GAMIFICATION

React 18 · TypeScript · Vite · Tailwind CSS · Radix UI · Monaco · React Flow · Recharts · Node.js · Express 5 · MongoDB Atlas · Mongoose 9 · JWT · bcrypt · Gemini · Groq · Ollama · pdf-parse · Vercel · Render

- ✓ XP, levels, streaks, badges + "solar-system" skill map
- ✓ Prerequisite-linked skill nodes; "Nebula Forge" AI challenge generator

CONCLUSION

CodeGalaxy is a working, full-stack programming-education platform with a real multi-language execution engine, a resilient three-provider LLM fallback chain, a lexical document-retrieval pipeline, a persistent gamification model and an AI interview simulator. Measured against its own design documents it is honestly AI-assisted rather than fully agentic today: a genuine planner-executor-synthesizer loop exists in code but is not yet reachable from any route.

LIMITATIONS & FUTURE WORK

Limitations: execution runs in a host child process (not container-isolated); document store is in-memory and single-instance; image Q&A depends on one Groq vision model; no automated test suite.

Future work: wire the agent loop into the repair endpoint; Docker / gVisor sandbox; Qdrant semantic search; SSE streaming; per-tenant rate limiting; offline-first sync.

Live demo: code-galaxy-an-agentic-ai-based-int.vercel.app

Source: github.com/HarshKumar5822/CodeGalaxy-An-Agentic-AI-Based-Interactive-Programming-Learning-Platform